Code development and submission in the MPhil for Scientific Computing

Philip Blakely

Centre for Scientific Computing, University of Cambridge

Part I

Programming in Practice

Suggested reading

- There are many books that claim to teach you how to program.
- They do not.
- They teach you the basics of a programming language.

Some books that teach good programming practice are:

- Code Complete - Steve McConnell
- Writing Solid Code - Steve Maguire

Light-relief (depending on your definitions of "light" and "relief"):

- `www.thedailywtf.com` (The Daily Worse Than Failure - demonstrating examples of poor programming practice in commercial software)

Text editor

A good text-editor can help reduce some bugs, such as matching `else` statements with the wrong `if`.

- The text-editor you use is a matter of personal taste.
- However, it should have automatic indentation and syntax highlighting.
- I recommend emacs if you have no previous preference.
- You may use vi/vim/etc. if you wish
- Alternatives include geany and Kate, both available on CSC machines.
- The editor Sublime exists, but requires a paid-for licence at $80.
- VS Code (Microsoft) works well, including with remote editing, although it fills up `/home` and runs weird processes on the remote node.

Avoid magic numbers

- Magic numbers are apparently arbitrary constants with no clear basis or documentation for their value.
- Consider which of the following lines is clearer, especially if updating from 2D to 3D:

```
double scaleFactor = pow(cellSize, 1.0/2.0);
double scaleFactor = pow(cellSize, 1.0/DIMN);
```

- Also, what if I type π from memory, and remember a different precision each time:

```
double one = pow(sin(3.14159/2), 2) +
             pow(cos(3.1415927/2), 2);
```

Now one $= 0.99999999999824018548$.

Containing magic numbers

You should create a file such as `constants.H`:

```
extern const double pi;
extern const unsigned int dimn;
extern const unsigned int maxWidgets;
```

and `constants.C`:

```
//! Value of pi to use (because I forgot about M_PI)
const double pi = 3.14159265358979;

//! Dimensionality of simulation
const unsigned int dimn = 2;

//! Maximum number of product IDs for widgets
const unsigned int maxWidgets = 1000;
```

Naming conventions

There are many conventions for naming variables and classes.
Choose your own (and stick to it).
Personally, I use:

Variables	`camelCaseNames`
Global/Constant Variables	`CapitalizedCamel`
Class names	`CapitalizedNames`
Class member data	`m_classData`
Functions	`myCamelFunction()`
Macros	`DIMENSION`

Settings file

- Avoid hard coding filenames etc. into the code.
- You (and the assessors) will be running your code more than once, for different sets of parameters.
- You could hard-code these parameters into your code:

```
const double domainMin = -1.0;
const double domainMax = +1.0;
```

  and recompile every time you want to use different parameters.
- However, you will want to reproduce your results later, reliably, and with minimum work.
- You should write your code to support input from a settings file.

Settings file

The settings file format could be as simple as:

```
-1.0    domainMinimum
+1.0    domainMaximum
```

where you could use the `ifstream.getline()` function:

```
#include <iostream>
#include <sstream>
#include <fstream>

std::ifstream file("settings.txt");
while(file){
  std::string valueStr;
  double value;
  std::getline(file, valueStr);
  if(!file) break;
  std::istringstream input(valueStr);
  input >> value;
  std::cout << "value = " << value << std::endl;
}
```

libconfig

A possible library for importing settings files is:
`http://www.hyperrealm.com/libconfig/libconfig.html`
This allows you to write settings files such as:

```
CFD: {
  domainMin = -1;
  domainMax = +1;
  EoS: {eos = "ideal-gas";gamma = 1.4; }
}
```

and read them using:

```
libconfig::Config cfg;
cfg.readFile("settings.txt");
const auto& cfd_section = cfg.lookup("CFD");
double domainMin, domainMax;
cfd_section.lookupValue("domainMin", domainMin);
cfd_section.lookupValue("domainMax", domainMax);
```

Commenting

Very bad comment:

```
// Check that "6" is in the vector
if(v[0] == 5)
```

Bad/pointless comment:

```
// Loop over all elements of v
for(size_t i=0 ; i < v.size() ; i++)
```

Good comment:

```
// Compute a second order finite-difference update
for(size_t i=1 ; i < v.size()-1 ; i++){
  w[i] = v[i] + ( f(v[i+1]) - f(v[i-1]) )*dt/dx;
}
```

Extended good comment:

```
/* Perform Gaussian elimination on the matrix A, and
   ensure that all zero rows are at the bottom.
   If we do not do this, then the algorithm in
   findEigenvaluesFast() will not work correctly. */
```

If a comment does not match the associated code, then both should be considered wrong until proved otherwise.

Testing

- When you make a change to your code, make sure it doesn't break previously working tests.
- Your settings files can be stored in version control, and run using a script (overnight if necessary).
- If you find a code-version is broken, use `git bisect` or a similar manual process to track down when the mistake was introduced.

Libraries

- You are stongly encouraged to use libraries where possible to reduce your workload, reduce errors, and (possibly) improve efficiency.
- Possible libraries include:
  - libconfig++
  - Armadillo `http://arma.sourceforge.net/`
  - Boost `http://www.boost.org`
  - Eigen `https://eigen.tuxfamily.org/dox/`
  - BLAS / LAPACK (`http://www.netlib.org/blas/`) (use implementations from reputable sources)
- This is by no means complete. Other students / researchers may have more suggestions.
- As a rule of thumb, try not to use libraries that have not been updated for over 2 years.
- If you find you need a particular library version, make a note of this for yourself and the assessors.
- This is especially true if bugs are fixed/introduced.

Version control

- I covered git in Michaelmas Term.
- Please use this (or an equivalent such as Subversion).
- However, when submitting code to the assessors, only submit the final code.
- Past students have occasionally submitted their entire git history, making for a much larger download for the assessors to check. (As well as revealing all your past commit messages, etc.)
- Do not add executables, object files, or PDFs to your repository. Again this makes for a large download for assessors.
- If using GitHub or similar, do not make your repository public. If you do, this would allow other students (current and future) to reuse your code, potentially leading to plagiarism investigations.

Library licensing

- For an MPhil (or Ph.D.), software licensing is unlikely to affect you.
- However, if you ever intend to commercialise your research, be aware that just because source-code is freely available to download from somewhere, you may not be able to legally redistribute it to others.
- This is especially the case if you ever distribute closed-source binaries.
- Then, you will need to understand the difference between GPLv2, GPLv3, LGPL, MIT, BSD, etc. licences.
- These may limit how you can use the libraries / code in commercial projects, and/or whether you may redistribute the source-code, even if you do so for no charge.
- If you have a public GitHub repository and put code in it that you do not have permission to redistribute, this could be a problem.

Modularisation

- The human brain can only hold a few pieces of data at once.
- Having to remember the status of a large number of variables when writing/debugging is hard and error-prone.
- Before starting to write your code, you should consider how to break it down into managable chunks.
- When writing your code, if you find yourself writing similar code again and again you should factor it out as a function.
- This will allow for easy optimization, debugging, or replacement by a library function.

Part II

Code submission

Code submission

- As part of both your written assignments and project report submissions, you are required to submit the code you have written to produce the results presented.
- This will be used by the assessors as material to support their assessment of your work.
- The code should be uploaded to Moodle as a `.tar.gz` file (or similar) at the same time as the assignment/project.
- The precise location will be sent to you by CSC admin nearer the time.

What should I submit?

- You should submit, as far as possible, only code and settings files you have written.
- You do not need to submit data files that are available elsewhere.
- You do not necessarily need to submit data generated by your source-code (or object files or executables).
- In Scientific Computing the amount of original code you can generate in 6 months is unlikely to be vast.
- The amount of original work (and hence data) you submit should probably be of the order of MB, not GB.
- Exceptions may exist for some data-processing projects where the data collection from diverse sources constitutes part of the research.

Avoiding plagiarism

- Where possible, do not include libraries in submitted code.
- Making use of pre-existing libraries which completely solve your research problem in one line is not plagiarism (if you reference them correctly), but does demonstrate to the assessors that you have not done any research and may not understand the algorithm.
- When the assessor or examiner goes through the code, they will be looking for code that is clearly in a different style from the rest of it, or copyright headers that have been left in.
- If you make use of someone else's code, you must reference it within your dissertation. If not, then this may count as plagiarism.

Plagiarism checking

- We will be using TurnItIn, which is designed to detect plagiarism in written academic work. This will be done in accordance with the University requirements, including that your name (or other identifying information) should not appear in the written work.
- For more information on the University use of TurnItIn, see: `https://www.plagiarism.admin.cam.ac.uk/investigating/turnitin/students`
- We also make use of MOSS `http://theory.stanford.edu/~aiken/moss/` which detects similarity between your and other students' code.

ChatGPT/DeepSeek/AI

- The University is obviously aware that ChatGPT, DeepSeek, and other A"I" tools exist.
- Given suitable prompts, these can produce superficially plausible answers to a wide range of questions, and produce plausible looking source-code.
- As their answers to standard problems are likely correct (having been regurgitated from widely available lecture notes and text-books), you may be lulled into a false sense of security.
- Unfortunately, their answers to questions in the less-standard assignments, and especially the research element are likely to contain errors.
- Further, their use of unacknowledged lecture-slides etc. means that any work produced will not be correctly referenced. Correct referencing is required in both W/A and research project.

ChatGPT/DeepSeek/AI

- In January 2026 I asked ChatGPT for C++ code for the SLIC finite-volume method for linear advection; it was better than January 2025. This year (unlike past years) I couldn't immediately fault it. However, when asked to refine the code beyond the initial work, it began to make some subtle mistakes around slope-limiting for Euler equations.
- There are many other problems with ChatGPT and similar tools, including systematic and prejudicial biases, as well as copyright and privacy issues due to the unclear origins of the large datasets the models have been trained on.
- It's much better use of your time to understand what you are writing, because ChatGPT (and similar LLMs) will be of very little use for your research project.
- Your research will need to be your own, so if you do not understand the algorithms/methodologies sufficiently, you will not perform well in this part of the assessment.

AI and Plagiarism

- Note that the University's plagiarism definition includes "Contract cheating: contracting a third party to provide work".
- Content produced by AI platforms, such as ChatGPT, would not be original work.
- So, using AI to generate text or source-code which is submitted for assessment is unacceptable, breaches the University's Rules of Behaviour and can result in serious academic sanctions.
- To quote: "A student using any unacknowledged content generated by artificial intelligence within a summative assessment as though it is their own work constitutes academic misconduct, unless explicitly stated otherwise in the assessment brief."
- Part of the purpose of the presentation, poster, and viva parts of the MPhil assessment is to check whether you have understood the work you have submitted for assessment. If this is not the case, it will be reflected in your marks.

Extending someone else's code

- If you have modified an existing package, then you should only submit the files that you have changed, indicating which parts you have modified by suitable commenting with your Blind Grading Number.

```
//! Added function to pre-compute pi - HGP23M
double calcPi();
```

- Obviously, you should reference the original code in your project.
- If the original code is not widely available, then it may be appropriate to include it with your submission.
- This may be the case for software unique to your research group or supervisor, but not for code publicly available on GitHub or elsewhere.
- A `README` file should indicate which files are (or contain) your own work.

Project coding guidelines

- The assessors should be able to rerun any simulations and generate any results presented in your dissertation.
- The presentation of your code is as important as the presentation of your dissertation.
- Instructions for compiling and running your code should be clear and complete, preferably in a `README` file.
- Remember that the assessors' OS may be different from yours (however you may assume they are using Linux).
- You may assume sensible/default locations for libraries in a `Makefile` or `CMakeLists.txt`, but should label them appropriately therein.
- You may not need to include all settings files used to generate results. However, a representative sample should be included, or another script to generate such files (for a parametric study, for example).

Black box codes

- If you are using an existing research code as a "black-box", with no modification, then you may think you have no source-code to submit.
- However, in this case you should upload any scripts that produce settings files for a parameter study, or any post-processing scripts that turn the output data into plots.
- For Machine Learning projects, you should upload your ML/PyTorch/etc. code that uses the simulation software as input.

Public funding and Open Data

- Anyone funded by research bodies should take note of their Open Access Data policy.
- They may require you to make publicly available (with a few exceptions) sufficient data to allow any published results to be reproduced and scrutinised.
- This includes source-code used in generating data, and perhaps the generated data as well.
- The data and source-code must be stored for at least 10 years (the University provides a suitable repository at `www.data.cam.ac.uk`).
- In order to be of any use to the scientific community and the wider public, it should be easy for them to compile and use your code without contacting you.
- This may be the time to consider the licensing and availability of any libraries used in your code.
- See `https://www.openresearch.cam.ac.uk/data/` or ask your supervisor for more information.

Finally...

- The guidelines in this presentation are just that. Guidelines.
- Ultimately, you need to satisfy the assessors that you have done the work that you present to them.
- If the assessors require further information to persuade themselves of this, they will ask for it.
- Due to the multi-disciplinary nature of the MPhil, these guidelines may not cover all projects.
- If you need further information, please ask...